Predicting Coronary Heart Disease from a Health Survey with Linear Models

Léon Lhuillier, [Teammate 2], [Teammate 3]
CS-433 Machine Learning, EPFL, Switzerland

***Abstract*—We predict whether a person has had a heart attack or coronary heart disease from 321 answers to the BRFSS 2015 telephone survey, using only linear models implemented in NumPy. Most of the work is in turning survey *codes* into meaningful numbers: decoding fields that pack a unit inside the answer, separating "refused" from "never", and one-hot encoding categories. Because only 8.8% of the people are positive, we optimize the decision threshold for F1. With an honest 5-fold cross-validation, in which every statistic and the threshold are fitted on training folds only, regularized logistic regression reaches an F1 of 0.426 ± 0.005, against 0.415 for a naive median-impute-and-standardize baseline.**

I. Introduction

The Behavioral Risk Factor Surveillance System (BRFSS) is a US telephone survey on health behaviours. We receive 328,135 labelled respondents and 109,379 test respondents, each described by 321 columns, and must predict a binary label (heart disease $+1$, otherwise -1). Predictions are scored with the F1 score, $F_1 = 2TP/(2TP + FP + FN)$. Since only 8.8% of training labels are positive, a classifier that always predicts -1 is 91.2% accurate but has $F_1 = 0$: accuracy is not a useful guide here.

The project imposes the six linear methods seen in class (least squares, ridge regression, gradient descent (GD) and stochastic GD on the mean squared error (MSE), logistic and regularized logistic regression) and forbids external libraries. Our contribution is therefore mainly a careful, reproducible preprocessing pipeline and a leakage-free validation protocol.

II. Data and preprocessing

Survey codes. Answers are codes, not measurements. Across most questions, 7/77/777 mean "don't know" and 9/99/999 "refused", which we map to missing (NaN), while 8/88/888 mean "none" or "never", which we map to 0. We wrote a metadata table (`data_columns.csv`) giving, for 263 columns, this value mapping, the valid range and whether the column is categorical. Values outside the valid range become NaN.

Special columns. Twenty columns pack a unit inside the number: `FRUIT1`$=205$ means "5 times per week", `EXERHMM1`$=130$ means 1 h 30, and `WEIGHT2` is in pounds unless it starts with 9 (kilograms). We decode them into a single unit per column (times per day, minutes, kg, cm, ...) *before* applying the value mapping, so that codes like 888 are still raw when decoded. A multiple-choice column (`PCDMDECN`, "12" = options 1 and 2) is split into one binary column per option.

Feature construction. The following steps are fitted on training data only (Table I). (1) We drop identifiers and interview metadata (record number, date, state, phone type, survey weights) and the 58 columns without metadata. (2) We drop columns with more than 90% missing values. Many questions are only asked to a sub-group (e.g. diabetics), so a high missing rate is common: the median column is 36% missing. (3) Categorical columns with 3 to 15 categories are one-hot encoded with the categories seen in training; otherwise a value such as "marital status $= 6$" would be treated as six times status 1. (4) For numerical columns, a $\log(1 + x)$ transform is applied to non-negative columns whose 99th percentile exceeds ten times the median (e.g. blood-sugar checks per year, up to 36,135). (5) Missing values are replaced by the training median, and a binary "was missing" indicator is added for every column with more than 1% missing values, since a skipped question carries information. (6) Columns are standardized with the training mean and standard deviation, which makes gradient descent well-conditioned and the ridge penalty treat all features equally. A bias column is added.

Table I
Features after preprocessing (full training set).

Block	Columns
Numerical (median-imputed, standardized)	139
of which $\log(1+x)$-transformed	16
One-hot (31 categorical variables)	145
Missing-value indicators	109
Bias	1
Total (constant columns removed)	394

III. Models and validation

Models. Linear models output a score $s = \tilde{x}^\top w$; we predict $+1$ when $s \geq t$. For MSE-based models we regress $y \in \{0, 1\}$; logistic models use the same labels. Regularized logistic regression minimizes the mean negative log-likelihood plus $\lambda\|w\|^2$ by full-batch gradient descent (step size $\gamma = 0.5$, 300 iterations, $w_0 = 0$). We also tested a

class-weighted variant in which positive samples weigh 3 or 6 times more.

Threshold. With 8.8% positives, almost no score reaches the default threshold of 0.5, which gives F1 ≈ 0.13. We therefore pick t by maximizing F1 over 200 quantiles of the training scores.

Cross-validation. We use 5-fold cross-validation (seed 1). In each fold, the whole pipeline (all preprocessing statistics, the weights and the threshold t) is fitted on the four training folds, and F1 is measured once on the held-out fold. Choosing t on the evaluated data, as in our first experiments, gives optimistic scores. We report the mean and standard deviation over folds.

IV. Results

Table II
5-FOLD CROSS-VALIDATED F1 (MEAN $\pm$ STD). "BASELINE": RAW CODES, MEDIAN IMPUTATION AND STANDARDIZATION ONLY. "OURS": PIPELINE OF SECTION II.

Model	Baseline	Ours
Least squares	0.397 ± 0.028	singular
Ridge, $\lambda = 10^{-4}$	0.411 ± 0.004	0.420 ± 0.006
Ridge, $\lambda = 10^{-2}$	0.410 ± 0.004	0.418 ± 0.006
Reg. logistic, $\lambda = 10^{-4}$	0.415 ± 0.004	$\mathbf{0.426 \pm 0.005}$
Weighted logistic, $\times 3$	0.415 ± 0.004	0.425 ± 0.005
Weighted logistic, $\times 6$	0.413 ± 0.004	0.422 ± 0.005

Table II shows three findings. First, preprocessing improves every model by about 0.01 F1, roughly twice the fold-to-fold standard deviation. Second, logistic regression beats the MSE-based models by about 0.006. Least squares is unstable on raw codes (std 0.028) and fails on our features: the one-hot blocks are linearly dependent, so $X^\top X$ is singular. Ridge regression removes the problem through the term λI. Third, class weighting does not help once the threshold is tuned: moving t already trades precision for recall, and weighting only shifts the scores.

Table III
ABLATION: 5-FOLD F1 OF RIDGE ($\lambda = 10^{-4}$) WHEN ONE STEP OF OUR PIPELINE IS REMOVED OR CHANGED (FULL PIPELINE: 0.420 ± 0.006).

Variant	F1	Features
Full pipeline	0.420 ± 0.006	394
No one-hot (categories as numbers)	0.417 ± 0.006	301
Drop columns $> 50\%$ missing (not $> 90\%$)	0.417 ± 0.007	324
No missing-value indicators	0.419 ± 0.007	285
No $\log(1 + x)$ transform	0.419 ± 0.007	394

The ablation (Table III) shows that one-hot encoding and keeping sparsely answered questions contribute most (about 0.003 each), while missing indicators and the log transform are within noise. No single step explains the gain on its own: it comes from cleaning the codes as a whole.

Our submission (`run.py`) refits the pipeline and the regularized logistic regression ($\lambda = 10^{-4}$) on the full training set, with the threshold chosen on the training scores.

V. Discussion

The decisive steps were cheap: tuning the threshold for F1 (from 0.13 to about 0.42), then making the features meaningful. With linear models and a noisy self-reported target, further gains are small, and differences below the fold standard deviation (≈ 0.005) should not be over-interpreted. The main limitations are the linear decision boundary, median imputation, which ignores relations between questions, and our hand-written metadata, which may contain coding mistakes. Natural next steps are interaction features between strong predictors (age, hypertension, diabetes), and models with a non-linear decision boundary.